

Bússola | Resumo para o grupo

Estado do projeto e testes realizados • 02/10/2026

1. O que já foi desenvolvido

Backend em Java 25 e Spring Boot, banco MySQL e frontend em HTML, CSS e JavaScript puro. A documentação do TCC foi estudada até a seção 2.4.1; a memória mantém o histórico e as divergências identificadas.

Foram criadas 12 entidades e seus repositories: usuário, decisão, opção, critério, avaliação, brainstorming, item GUT, pró/contra, reflexão 10/10/10, inversão, resultado e token de recuperação de senha. Essas estruturas permitem persistência, mas não significam que todos os fluxos de negócio estão implementados.

A migração V1 cria tabelas, relacionamentos, índices e restrições. O Flyway controla a aplicação da migração; o Hibernate valida a compatibilidade das entidades com o banco.

2. Cadastro e frontend atual

O POST /auth/cadastro recebe nome, nascimento, e-mail, senha e aceite dos termos. Valida os dados, exige idade mínima de 16 anos, normaliza o e-mail, verifica duplicidade, registra o aceite e salva somente o hash BCrypt da senha, com custo 10.

Sucesso: HTTP 201, com id, nome e e-mail. Dados inválidos: HTTP 400. E-mail duplicado: HTTP 409. A resposta não contém senha nem hash. A rota é pública e a configuração CORS permite o frontend local na porta 8765.

Já existem página inicial, Sobre, cadastro e Home provisória. O JavaScript do cadastro valida o formulário, envia dados à API, apresenta erros e redireciona à Home quando recebe 201. Há exibição/ocultação da senha e modais de termos e privacidade. Esse redirecionamento não implementa login ou sessão autenticada.

O Swagger documenta a API e permite chamadas reais pelo navegador:

<http://localhost:8080/swagger-ui/index.html>. Usar Execute na aplicação real pode gravar dados no banco configurado.

3. Como os testes funcionam

Os testes seguem três passos: preparar um cenário, executar a ação e conferir o resultado. O JUnit executa os testes; o Mockito simula dependências; o MockMvc verifica requisições e respostas da camada web sem precisar abrir um navegador.

- **Serviço:** regra de negócio real, repository simulado e BCrypt real. Não utiliza MySQL.
- **Controller:** validação HTTP e segurança da rota, com serviço simulado. Não utiliza MySQL.
- **Integração:** aplicação e persistência com MySQL real, para verificar a comunicação entre componentes.

Testes e execução em equipe

4. Cenários verificados e resultados anteriores

- **5 testes de serviço:** cadastro aos 16 anos completos; rejeição abaixo da idade; e-mail duplicado; falta de aceite; nome em branco. O cenário válido também confere normalização, hash BCrypt/custo 10 e registro do aceite. Os cenários inválidos verificam que não se tenta salvar.
- **5 testes de controller:** cadastro público com 201 e sem senha/hash; e-mail inválido; termos não aceitos; tradução da exceção de idade para 400; tradução da duplicidade para 409. No controller, a idade é simulada pelo serviço; o cálculo real é testado no serviço.
- **2 testes de Swagger:** acesso público ao HTML e à configuração; documentação do endpoint com exemplo de e-mail. Não verificam cliques no navegador.
- **2 testes temporários de banco:** tabelas e persistência de entidades, relacionamentos e códigos dos enums. Flyway aplicou a V1 e Hibernate validou os mapeamentos em MySQL separado na porta 3307. Os testes e a instância temporária foram removidos após a verificação.

Resultados registrados: os 12 testes de serviço/controller/Swagger passaram nas rodadas anteriores, sem falhas, erros ou ignorados. Os 2 testes temporários de MySQL também passaram. Houve ainda um cadastro manual real, confirmado no Workbench com senha armazenada em BCrypt.

O teste original contextLoads() tenta iniciar o contexto completo do Spring e depende do banco. Ele não foi executado nas últimas rodadas descritas. Nenhum teste foi reexecutado para produzir este resumo; os resultados são históricos e não certificam alterações posteriores ou todos os cenários possíveis.

5. Como executar e compartilhar o projeto

Consultar **COMO_RODAR.md** na raiz para configuração completa. Requisitos: Git, MySQL 8, JDK 25, Python 3 e navegador; o backend possui Maven Wrapper. Portas locais: MySQL 3306, backend 8080 e frontend 8765.

Configurar DB_URL, DB_USERNAME e DB_PASSWORD no ambiente que inicia o backend. No PowerShell, as variáveis valem para aquela janela. Iniciar o backend na pasta backend com **.lmvnw.cmd spring-boot:run**; em outro terminal, na raiz, iniciar **.lrodar-frontend.ps1**. Manter os terminais abertos. Acesso:

<http://127.0.0.1:8765/pages/index.html>.

Atualização de Júlia: a senha do MySQL local foi resolvida. Usar a senha local em DB_PASSWORD; não compartilhar seu valor. Essa informação não representa uma nova execução ou teste de conexão nesta tarefa. Cada integrante configura as credenciais do próprio computador.

Teste visual de uma tela sem chamadas à API pode exigir apenas frontend. Para testar Criar conta de verdade, manter MySQL + backend + frontend disponíveis. Chamadas pelo Swagger exigem backend e banco para o cadastro.

6. Regras do grupo e próximos passos

Ao concluir uma tarefa, verificar impactos na execução e nos testes. Atualizar COMO_RODAR.md quando houver instruções novas; atualizar rodar-frontend.ps1 somente quando mudar a inicialização ou atalhos úteis. Ler as versões atuais, preservar melhorias do grupo, explicar os passos para iniciantes e nunca incluir credenciais ou secrets.

Trabalhar na branch Julia. Quando aplicável, executar git diff --check e git status --short, relatar os arquivos alterados e o impacto na execução/documentação. Commit, push e PR somente mediante pedido explícito.

Ainda faltam login/JWT e sessão, recuperação de senha funcional, fluxo de decisões e etapas, técnicas de análise e cálculo, finalização, histórico e exclusão. A existência das entidades e da Home provisória não comprova esses fluxos prontos.